happening here. In this case, I decided to focus on the essential aspects of using the functional programming paradigm in the context of an event-driven application. Inside the function, you can add any processing logic you like. What matters here is the contract provided by the function, its signature: inputs and outputs. After defining that, you're free to implement the function as needed. I could have provided a more real-world implementation of this function, but it would have added nothing valuable considering the goal of this chapter. It doesn't even have to be Spring-based code. And in this example, it's not: it's plain Java code.

Spring Cloud Function is capable of managing functions defined in different ways, as long as they adhere to the standard Java interfaces `Function`, `Supplier`, and `Consumer`. You can make Spring Cloud Function aware of your functions by registering them as beans. Go ahead and do that for the `pack()` function by annotating the `DispatchingFunctions` class as `@Configuration` and the method as `@Bean`.

**Listing 10.6 Configuring the function as a bean**

```
@Configuration                                                    ❶
public class DispatchingFunctions {
  private static final Logger log =
    LoggerFactory.getLogger(DispatchingFunctions.class);

  @Bean                                                           ❷
  public Function<OrderAcceptedMessage, Long> pack() {
    return orderAcceptedMessage -> {
      log.info("The order with id {} is packed.",
        orderAcceptedMessage.orderId());
      return orderAcceptedMessage.orderId();
    };
  }
}
```

❶ Functions are defined in a configuration class.

❷ Functions defined as beans can be discovered and managed by Spring Cloud Function.